Tarea 5 - Estructuras de Datos, Divide y Vencerás

Análisis y Diseño de Algoritmos — Maestría PCIC (Semestre 2027-1)

Tlacaelel Jaime Flores Villaseñor Victor Rosales Jaimes

2026-11-08

1 Ejercicio 1. Heap binario máximo y heapsort

1.1 Enunciado

Considera los algoritmos de heap binario máximo y heapsort que aparecen en las diapositivas utilizadas en clase.

1.1.1 Inciso a. Propiedad de `sink`

Demuestra la siguiente propiedad de la función `sink` (diapositiva 20): si al llamar `sink(A, k, n)`, cada uno de los subárboles con raíces $A[2k]$ y $A[2k+1]$, respectivamente, cumple con la propiedad heap-max (es decir, el entero en cada vértice del árbol es mayor o igual que los enteros en sus hijos), entonces al finalizar `sink`, el subárbol con raíz $A[k]$ cumple con la propiedad heap-max.

1.2 Solución del inciso a

1.2.1 Funcionamiento de `sink`

La función `sink(A, k, n)` restaura la propiedad heap-max en el subárbol cuya raíz está en la posición k, suponiendo que los subárboles de sus hijos ya cumplen dicha propiedad. Mientras el nodo actual tenga al menos un hijo, se selecciona al hijo de mayor valor y se compara con el padre. Si el padre es mayor o igual que ese hijo, el procedimiento termina, pues también es mayor o igual que cualquier otro hijo existente. En caso contrario, se intercambian ambos valores y se continúa desde la posición del hijo seleccionado, a la que descendió el valor del padre. Este proceso se repite hasta llegar a una hoja o hasta que el nodo actual cumpla la propiedad heap-max con sus hijos. Al finalizar, todo el subárbol de la posición inicial satisface dicha propiedad.

1.2.2 Demostración

La demostracion se hace por inducción.

- Caso base: Si $h = 0$ el subarbol consiste únicamente en una hoja, como esta no tiene hijos cumple con la propiedad de heap-max y `sink` termina sin realizar cambios.
- Hipótesis de induccion: Suponemos que la afirmación es cierta para todo los subárboles con altura menor a h.
- Paso inductivo: Consideremos un subárbol T de altura $h > 0$, cuya raíz está en la posición k y contiene el valor $x = A[k]$. Por la condición del enunciado, los subárboles que comienzan en los hijos de esta raíz ya cumplen la propiedad heap-max.

Sea j la posición del hijo de mayor valor y sea $y = A[j]$. Si la raíz tiene un solo hijo, seleccionamos ese hijo. Entonces tenemos dos casos:

1. Si $x \geq y$, la raíz es mayor o igual que cada uno de sus hijos. Como los subárboles de estos ya son heaps máximos, todo T cumple la propiedad heap-max y `sink` termina.
2. Si $x < y$, el algoritmo intercambia ambos valores: y queda en la raíz de T y x queda en la posición j y el procedimiento continúa desde j.

En el segundo caso, consideremos el subárbol cuya raíz está en j. Su altura es menor que h. El intercambio solo cambió el valor de su raíz; los subárboles de los hijos de j permanecieron intactos y siguen siendo heaps máximos. Por lo tanto, por la hipótesis de inducción, dicho subárbol queda convertido en un heap-max.

Falta comprobar que el valor y, que subió a la raíz de T, queda correctamente colocado. Antes del intercambio, y era el máximo de su propio subárbol. Además, si existe otro hijo de la raíz de T, y es mayor o igual que su valor y, por tanto, que todos los valores de ese otro subárbol. Como también $y > x$, resulta que y es mayor o igual que todos los valores que quedan debajo de él. Los intercambios posteriores únicamente reorganizan esos valores, por lo que esta relación se conserva.

Así, al finalizar, los subárboles de ambos hijos existentes son heap-max y la raíz de T es mayor o igual que sus hijos. Por consiguiente, todo T cumple la propiedad heap-max.

1.3 Enunciado del inciso b

Demuestra la siguiente invariante de ciclo de la fase de construcción de heapsort (ciclo de la diapositiva 42): al inicio del ciclo, para todo $k + 1 \leq \ell \leq n$, se cumple que el subárbol con raíz en $A[\ell]$ cumple con la propiedad heap-max.

1.4 Solución del inciso b

Demostraremos que, al inicio de cada iteración, para toda posición ℓ tal que $k + 1 \leq \ell \leq n$, el subárbol con raíz en $A[\ell]$ cumple la propiedad heap-max.

1.4.1 Inicialización

Antes de la primera iteración, $k = \lfloor n/2 \rfloor$. Todas las posiciones $\ell > \lfloor n/2 \rfloor$ corresponden a hojas, pues la posición de su posible hijo izquierdo, 2ℓ, es mayor que n. Cada hoja cumple la propiedad heap-max porque no tiene hijos. Por tanto, el invariante se cumple al inicio de la primera iteración.

1.4.2 Mantenimiento

Supongamos que el invariante se cumple al inicio de una iteración cuyo índice es k. Los hijos existentes del nodo en la posición k están en las posiciones $2k$ y $2k + 1$, ambas mayores que k. Por el invariante, los subárboles con raíz en esos hijos cumplen la propiedad heap-max.

En consecuencia, se cumplen las condiciones del inciso a: después de ejecutar sink(A,k,n), todo el subárbol con raíz en la posición k es un heap máximo.

Además, los subárboles que comienzan en posiciones mayores que k siguen cumpliendo la propiedad. Los que están dentro del subárbol recién reparado la cumplen porque forman parte de un heap máximo; los que están fuera permanecen sin cambios.

Así, después de ejecutar sink, todos los subárboles con raíz en posiciones ℓ tales que $k \leq \ell \leq n$ cumplen la propiedad heap-max. Al disminuir el índice a $k' = k - 1$, este intervalo equivale a $k' + 1 \leq \ell \leq n$. Por tanto, el invariante se conserva para la siguiente iteración.

1.4.3 Conclusión al terminar

El ciclo termina después de procesar $k = 1$ y disminuir el índice a $k = 0$. Por el invariante, todos los subárboles con raíz en posiciones $1 \leq \ell \leq n$ cumplen la propiedad heap-max. En particular, la cumple el subárbol con raíz en la posición 1, que contiene todo el árbol. Por lo tanto, al finalizar la fase de construcción, el arreglo representa un heap máximo.

2 Ejercicio 2. Emparejamiento de estaciones en un árbol

2.1 Enunciado

Sea T un árbol binario enraizado. Todos los vértices de T tienen un atributo binario `flag`, y solo para un número par de ellos, a los que llamaremos **estaciones**, su atributo es `true`.

Se busca emparejar las estaciones de forma tal que en T existan caminos disjuntos en aristas que conecten las parejas de estaciones. Los caminos no pueden compartir aristas, aunque sí pueden compartir vértices.

Diseña un algoritmo tipo divide y vencerás de tiempo $O(n)$ que imprima parejas de estaciones con la propiedad descrita, donde n es el número de vértices de T. Toda estación debe quedar emparejada.

Puedes suponer que existe una instrucción `Print(s)` que imprime la cadena `s` y toma tiempo $O(1)$. La raíz r de T puede ser parte de la entrada.

El árbol está representado como una lista de adyacencia. Para cada vértice v, left(v) y right(v) denotan sus hijos izquierdo y derecho, y $\pi(v)$ denota su padre. Si alguno no existe, su valor es `NONE`.

Explica la idea del algoritmo y da pruebas de terminación, corrección, tiempo y espacio. ¿Existen entradas para las que tu algoritmo corre en tiempo $O(\log n)$? Argumenta tu respuesta.

2.2 Idea del algoritmo

La idea del algoritmo es aplicar divide y vencerás sobre el árbol con raíz r. Para cada nodo v, se resuelven recursivamente los subárboles de sus hijos. Cada llamada devuelve, como máximo, una estación pendiente de emparejar, o `NONE` si todas las estaciones de su subárbol ya quedaron emparejadas. El nodo v reúne las estaciones pendientes recibidas y se incluye a sí mismo si es una estación. Con estas candidatas forma e imprime las parejas posibles y devuelve al padre la estación que quede pendiente, si existe. Durante este proceso se garantiza que los caminos de las parejas formadas no comparten aristas y que el camino desde la estación pendiente hasta v queda libre de aristas utilizadas por esas parejas. Como el árbol completo contiene un número par de estaciones, al finalizar la llamada sobre r no queda ninguna pendiente y todas quedan emparejadas.

2.3 Pseudocódigo

```
Función Emparejar(v)
    si v = NONE entonces
        devolver NONE

    izquierda ← Emparejar(left(v))
    derecha   ← Emparejar(right(v))

    P ← lista vacía

    si izquierda  NONE entonces
        AgregarAlFinal(P, izquierda)

    si derecha  NONE entonces
        AgregarAlFinal(P, derecha)

    si flag(v) = true entonces
        AgregarAlFinal(P, v)

    si longitud(P)  2 entonces
        Print((P[1], P[2]))
```

```
    si longitud(P) = 1 entonces
        devolver P[1]

    si longitud(P) = 3 entonces
        devolver P[3]

    devolver NONE
```

2.4 Prueba de terminación

Consideremos como medida el número de vértices del subárbol recibido por cada llamada a `Emparejar(v)`. Si $v =$ `NONE`, el subárbol está vacío y la función termina inmediatamente.

Si $v \neq$ `NONE`, el algoritmo realiza llamadas recursivas sobre los subárboles de sus hijos. Cada uno contiene estrictamente menos vértices que el subárbol con raíz en v, pues no incluye al propio v. Por tanto, la medida disminuye estrictamente en cada descenso recursivo y, al ser un entero no negativo, necesariamente se alcanza el caso base.

Una vez que regresan ambas llamadas recursivas, se ejecuta una cantidad finita de operaciones sobre una lista de tamaño máximo tres y se devuelve un resultado. En consecuencia, todas las llamadas terminan y, en particular, termina la llamada inicial `Emparejar(r)`.

2.5 Prueba de corrección

Demostraremos por inducción sobre el número de vértices del subárbol que, al finalizar `Emparejar(v)`, se cumplen las siguientes propiedades:

1. Todas las estaciones del subárbol quedan emparejadas exactamente una vez, salvo, como máximo, una estación pendiente. La función devuelve esa estación si existe y `NONE` en caso contrario.
2. Los caminos que conectan las parejas impresas son disjuntos en aristas.
3. Si se devuelve una estación pendiente, el camino desde ella hasta v no comparte aristas con los caminos de las parejas impresas.

2.5.1 Caso base

Si el subárbol está vacío, entonces $v =$ `NONE`. No hay estaciones ni caminos, y el algoritmo devuelve `NONE`. Por tanto, las tres propiedades se cumplen.

2.5.2 Hipótesis inductiva

Supongamos que las tres propiedades se cumplen para cualquier subárbol con menos de n vértices.

2.5.3 Paso inductivo

Consideremos un subárbol con raíz en v y $n > 0$ vértices. Cada subárbol hijo tiene menos de n vértices, por lo que podemos aplicar la hipótesis inductiva a ambas llamadas recursivas.

En consecuencia, cada llamada empareja todas las estaciones de su subárbol salvo, como máximo, una estación pendiente. Además, sus caminos son disjuntos en aristas y, si devuelve una estación, deja libre el camino que conecta esa estación con la raíz del subárbol hijo.

El algoritmo reúne en la lista P las estaciones pendientes recibidas y agrega a v si es una estación. La lista contiene como máximo tres estaciones distintas.

Emparejamiento de las estaciones. Las estaciones que no aparecen en P ya quedaron emparejadas exactamente una vez por las llamadas recursivas. Las que aparecen en P todavía no tienen pareja. El algoritmo procede según su cantidad:

- Si P está vacía, devuelve `NONE`.
- Si contiene una estación, la devuelve como pendiente.
- Si contiene dos estaciones, imprime su pareja y devuelve `NONE`.
- Si contiene tres estaciones, imprime la pareja formada por las dos primeras y devuelve la tercera.

Así, ninguna estación se empareja dos veces y queda pendiente, como máximo, la estación devuelta. Se cumple la primera propiedad.

Separación de los caminos en aristas. Por la hipótesis inductiva, los caminos impresos dentro de cada subárbol hijo son disjuntos en aristas. Los caminos de distintos subárboles hijos tampoco comparten aristas, pues dichos subárboles no comparten vértices.

Para cada estación pendiente recibida de un hijo, consideremos su camino hasta v. Este se obtiene extendiendo el camino libre garantizado por la hipótesis inductiva con la arista que conecta al hijo con v. Esa arista no fue utilizada por las parejas formadas dentro del subárbol hijo. Si v es una estación candidata, su camino hasta sí mismo no contiene aristas.

Los caminos de las distintas candidatas hasta v solo pueden coincidir en v, porque hay como máximo una candidata de cada subárbol hijo. Por tanto, al unir los caminos correspondientes a las dos estaciones emparejadas, se obtiene un camino que no comparte aristas con ninguna pareja previamente impresa. Se cumple la segunda propiedad.

Si queda una candidata pendiente, su camino hasta v tampoco comparte aristas con el de la nueva pareja ni con los de las parejas anteriores. Por ello, también se cumple la tercera propiedad.

Por inducción, las tres propiedades se cumplen al finalizar `Emparejar(r)`. Como cada pareja contiene dos estaciones, el número total de estaciones es igual al doble del número de parejas impresas más el número de estaciones pendientes. El total de estaciones es par y solo puede quedar cero o una pendiente. Por tanto, necesariamente quedan cero estaciones pendientes. En consecuencia, todas las estaciones quedan emparejadas exactamente una vez y los caminos que conectan sus parejas son disjuntos en aristas.

2.6 Tiempo de ejecución

Sea n el número de vértices del árbol. En cada llamada sobre un vértice v, el algoritmo procesa recursivamente sus dos subárboles y realiza una cantidad constante de trabajo adicional: construye una lista de tamaño máximo tres, imprime como máximo una pareja y devuelve una estación o `NONE`. Por el enunciado, cada impresión cuesta $O(1)$.

Si los subárboles izquierdo y derecho tienen n_{izq} y n_{der} vértices, respectivamente, entonces:

$$n_{\text{izq}} + n_{\text{der}} = n - 1.$$

Por tanto, el tiempo satisface la recurrencia:

$$T(n) = T(n_{\text{izq}}) + T(n_{\text{der}}) + \Theta(1), \qquad T(0) = \Theta(1).$$

Para resolverla, consideremos el árbol de llamadas recursivas. Cada vértice del árbol original genera exactamente una llamada no vacía, por lo que existen n llamadas de este tipo.

Además, cada llamada no vacía realiza dos llamadas sobre sus hijos, incluyendo aquellos que son `NONE`. Así, contando también la llamada inicial, el número total de llamadas es $2n + 1$. En consecuencia, hay $n + 1$ llamadas sobre subárboles vacíos.

Como cada llamada realiza trabajo local constante, al sumar los costos de todo el árbol de recursión obtenemos:

$$T(n) = \Theta(n) + \Theta(n + 1) = \Theta(n).$$

Por lo tanto, el algoritmo tiene tiempo de ejecución $\Theta(n)$ y, en particular, cumple la cota requerida de $O(n)$. Este análisis se aplica tanto a árboles balanceados como a árboles desbalanceados.

2.7 Espacio utilizado

Cada llamada a `Emparejar(v)` utiliza espacio constante para sus variables locales y para la lista P, que contiene como máximo tres estaciones. El árbol se consulta directamente, sin copiar sus subárboles, y las parejas se imprimen conforme se forman, por lo que no se almacenan en una estructura adicional.

El espacio auxiliar depende principalmente de la pila de recursión. Como las llamadas sobre los hijos se ejecutan secuencialmente, las llamadas activas corresponden a un camino desde la raíz hacia un descendiente. Si h es la altura del árbol medida en aristas, hay como máximo $h + 2$ llamadas activas, incluyendo una posible llamada con argumento `NONE`.

Por tanto, el espacio auxiliar es:

$$S(n) = O(h + 1).$$

Si el árbol está balanceado, su altura es $O(\log n)$ y el espacio auxiliar es $O(\log n)$. En el peor caso, el árbol puede ser una cadena de n vértices, con altura $n - 1$, por lo que el espacio auxiliar es $O(n)$. Esta cota excluye el almacenamiento del árbol de entrada.

2.8 ¿Existen entradas con tiempo $O(\log n)$?

No existen familias de entradas de tamaño creciente para las que el algoritmo propuesto tenga tiempo $O(\log n)$, pues siempre visita los (n) vértices del árbol y realiza trabajo constante en cada uno. Por tanto, su tiempo de ejecución es $\Theta(n)$, independientemente de la forma del árbol y de la distribución de las estaciones.

Incluso si el árbol está balanceado y tiene altura $O(\log n)$, el algoritmo procesa ambos subárboles de cada nodo. La altura determina la profundidad de la recursión, pero el tiempo total depende del trabajo acumulado en todas las llamadas, que es $(\Theta(n))$.

3 Ejercicio 3. Reconstrucción de un árbol binario

3.1 Enunciado

Considera árboles binarios enraizados, representados como en el ejercicio anterior. En un recorrido se realiza una exploración tipo DFS desde la raíz y se produce una secuencia en la que cada vértice aparece una y solo una vez. Las siguientes políticas se aplican recursivamente:

- **En orden:** subárbol izquierdo, raíz y subárbol derecho.
- **Preorden:** raíz, subárbol izquierdo y subárbol derecho.
- **Postorden:** subárbol izquierdo, subárbol derecho y raíz.

Se reciben un entero n y dos arreglos de longitud n, EO y PO, que contienen los recorridos en orden y en preorden, respectivamente, de un árbol binario enraizado T. El árbol no tiene por qué ser completo ni balanceado.

Diseña un algoritmo tipo divide y vencerás de tiempo $O(n^2)$ que produzca T a partir de n, EO y PO. Explica su idea y da pruebas de terminación, corrección, tiempo y espacio.

¿Existen entradas para las que tu algoritmo corre en tiempo $O(n \log n)$? ¿Y $O(n)$? ¿Cómo modificarías tu algoritmo si PO tuviera el recorrido en postorden en lugar de preorden? Argumenta tus respuestas.

3.2 Idea del algoritmo

La idea es reconstruir el árbol mediante divide y vencerás, utilizando los recorridos en orden EO y en preorden PO. En cada llamada, el primer elemento del segmento correspondiente de PO identifica la raíz del subárbol que se desea construir. Se busca ese vértice en el segmento correspondiente de EO: los elementos anteriores pertenecen al subárbol izquierdo y los posteriores al derecho. El número de elementos de cada lado permite delimitar también sus segmentos en PO, donde después de la raíz aparecen primero los vértices del subárbol izquierdo y luego los del derecho. Con esta información se crea la raíz y se reconstruyen recursivamente ambos subárboles, enlazándolos como sus hijos y estableciendo sus referencias al padre. Si el segmento está vacío, se devuelve `NONE`. Para evitar copiar los arreglos, cada llamada trabaja con índices que delimitan los segmentos. La llamada inicial procesa ambos recorridos completos y devuelve la raíz del árbol reconstruido.

3.3 Pseudocódigo

```
Función Reconstruir(EO, PO, i, j, p, padre)
    si i > j entonces
        devolver NONE

    valorRaiz ← PO[p]

    q ← i
    mientras EO[q]   valorRaiz hacer
        q ← q + 1

    tamañoIzquierdo ← q - i

    v ← CrearNodo(valorRaiz)
     (v) ← padre

    left(v) ← Reconstruir(
        EO, PO, i, q - 1, p + 1, v
    )

    right(v) ← Reconstruir(
        EO, PO, q + 1, j, p + 1 + tamañoIzquierdo, v
    )

    devolver v
```

3.4 Prueba de terminación

Consideremos como medida el número de elementos del segmento $EO[i..j]$. Si $i > j$, el segmento está vacío y la función devuelve `NONE` inmediatamente.

Si el segmento no está vacío, el ciclo de búsqueda incrementa q hasta encontrar `valorRaiz`. Como los arreglos contienen recorridos válidos del mismo subárbol, este valor aparece en $EO[i..j]$; por tanto, la búsqueda termina.

La posición encontrada q divide el segmento en $EO[i..q-1]$ y $EO[q+1..j]$. Ambos contienen estrictamente menos elementos que el segmento original, pues excluyen la raíz. En consecuencia, la medida disminuye estrictamente en cada descenso recursivo y, al ser un entero no negativo, necesariamente se alcanza el caso base.

Una vez que terminan ambas llamadas recursivas, solo se realizan las asignaciones de los hijos y se devuelve el nodo creado. Por lo tanto, todas las llamadas terminan y, en particular, termina la llamada inicial que reconstruye el árbol completo.

3.5 Prueba de corrección

Demostraremos por inducción sobre el número m de vértices del subárbol que `Reconstruir(EO, PO, i, j, p, padre)` devuelve su raíz y reconstruye correctamente sus vértices, sus hijos y sus referencias al padre. Suponemos que $EO[i..j]$ y el segmento de PO que comienza en p, de longitud m, son recorridos válidos del mismo subárbol.

3.5.1 Caso base

Si $m = 0$, el segmento está vacío y $i > j$. El algoritmo devuelve `NONE`, que representa correctamente un subárbol vacío.

3.5.2 Hipótesis inductiva

Supongamos que el algoritmo reconstruye correctamente cualquier subárbol con menos de m vértices a partir de sus recorridos en orden y en preorden.

3.5.3 Paso inductivo

Consideremos un subárbol de $m > 0$ vértices. Por la definición de preorden, su raíz es el primer vértice de su recorrido, es decir, $PO[p]$. Por tanto, el algoritmo identifica correctamente la raíz al asignar `valorRaiz ← PO[p]`.

Sea q la posición de ese vértice en $EO[i..j]$. Por la definición del recorrido en orden, los vértices del subárbol izquierdo son exactamente los de $EO[i..q-1]$, mientras que los del derecho son los de $EO[q+1..j]$. Así, el subárbol izquierdo tiene $q - i$ vértices.

En preorden, después de la raíz aparecen todos los vértices del subárbol izquierdo y luego los del derecho. Por ello, el preorden izquierdo comienza en $p + 1$ y el derecho comienza en $p + 1 + (q - i)$. En consecuencia, ambas llamadas recursivas reciben los segmentos que corresponden a sus respectivos subárboles.

Como cada subárbol hijo tiene menos de m vértices, por la hipótesis inductiva ambas llamadas los reconstruyen correctamente y devuelven sus raíces, o `NONE` si están vacíos. El algoritmo asigna estos resultados a `left(v)` y `right(v)`, respectivamente. Además, establece $\pi(v) =$ `padre` y pasa v como padre a ambas llamadas, de modo que las referencias al padre también quedan correctamente definidas.

Por tanto, el nodo creado y los subárboles reconstruidos forman exactamente el subárbol original. Esto completa el paso inductivo. Por lo tanto, la llamada inicial `Reconstruir(EO, PO, 1, n, 1, NONE)` reconstruye correctamente todo el árbol y devuelve su raíz r, cuya referencia al padre es `NONE`.

3.6 Tiempo de ejecución

Consideremos una llamada que reconstruye un subárbol con m vértices. Sean a y b los tamaños de sus subárboles izquierdo y derecho, respectivamente, de modo que:

$$a + b = m - 1.$$

La raíz aparece después de los a elementos del subárbol izquierdo en el segmento de EO. Como la búsqueda comienza al inicio del segmento, realiza $a + 1$ comparaciones. Crear el nodo, calcular los índices y establecer sus referencias requiere tiempo constante. Además, los arreglos no se copian.

Por tanto, el tiempo de una llamada no vacía, incluyendo sus llamadas recursivas, se expresa como:

$$T(m) = T(a) + T(b) + \Theta(a + 1), \qquad T(0) = \Theta(1).$$

Los valores de a y b dependen de la forma del árbol.

3.6.1 Peor caso

El peor caso ocurre, por ejemplo, cuando el árbol es una cadena de hijos izquierdos. En cada llamada, la raíz está al final del segmento de EO, por lo que se examinan todos sus elementos. Además, el único subárbol no vacío tiene un vértice menos:

$$T(m) = T(m-1) + \Theta(m).$$

Al desarrollar la recurrencia, obtenemos:

$$T(n) = \Theta(n) + \Theta(n-1) + \cdots + \Theta(1) = \Theta\left(\frac{n(n+1)}{2}\right) = \Theta(n^2).$$

Por otra parte, para cualquier entrada hay n llamadas no vacías y cada una realiza como máximo $O(n)$ trabajo local. Las llamadas sobre segmentos vacíos añaden $O(n)$ trabajo. Por tanto, ninguna entrada supera la cota $O(n^2)$, y la familia anterior demuestra que esta cota se alcanza.

3.7 Espacio utilizado

Cada llamada almacena una cantidad constante de variables y referencias. Como se utilizan índices, no se crean copias de los segmentos de los arreglos.

Si h es la altura del árbol, medida en aristas, la pila contiene como máximo $h + 2$ llamadas simultáneas, incluyendo una posible llamada sobre un segmento vacío. Por tanto, el espacio auxiliar es:

$$S(n) = O(h + 1).$$

En un árbol balanceado, este espacio es $O(\log n)$. En el peor caso, cuando el árbol es una cadena, es $O(n)$.

Además, el árbol producido contiene n nodos y requiere $\Theta(n)$ espacio. Si se incluye el almacenamiento de la salida, el espacio adicional total es $\Theta(n)$.

3.8 Entradas con tiempo $O(n \log n)$

Sí. Consideremos la familia de árboles binarios perfectos, en los que todos los niveles están completos. En cada llamada no trivial, los dos subárboles tienen el mismo tamaño y la búsqueda de la raíz requiere tiempo proporcional al tamaño del segmento. Así:

$$T(m) = 2T\left(\frac{m-1}{2}\right) + \Theta(m).$$

En el árbol de recursión hay $O(\log n)$ niveles. El trabajo acumulado en cada nivel es $O(n)$, pues los segmentos procesados en ese nivel son disjuntos. Por tanto:

$$T(n) = O(n \log n).$$

De hecho, para esta familia el tiempo es $\Theta(n \log n)$.

3.9 Entradas con tiempo $O(n)$

Sí. Consideremos la familia de árboles que forman una cadena de hijos derechos. Por ejemplo, para vértices identificados por $1, 2, \ldots, n$, sus recorridos son:

$$EO = [1, 2, \ldots, n], \qquad PO = [1, 2, \ldots, n].$$

En cada llamada, la raíz aparece al inicio del segmento de EO, por lo que la búsqueda realiza una sola comparación. El subárbol izquierdo está vacío y el derecho contiene un vértice menos. En consecuencia:

$$T(m) = T(m-1) + \Theta(1).$$

Al desarrollar la recurrencia, se acumulan n términos constantes, por lo que:

$$T(n) = \Theta(n).$$

Por tanto, para esta familia el algoritmo tiene tiempo $O(n)$.

3.10 Modificación para usar postorden

En postorden, los vértices aparecen en el siguiente orden: subárbol izquierdo, subárbol derecho y raíz. Por tanto, si el segmento de EO tiene $m = j - i + 1$ elementos y el segmento correspondiente de PO comienza en p, la raíz se encuentra en:

$$PO[p + m - 1].$$

Al localizar la raíz en la posición q de EO, obtenemos el tamaño del subárbol izquierdo, $a = q - i$. Como la raíz está al final del postorden, el segmento izquierdo comienza en p y el derecho en $p + a$.

3.10.1 Pseudocódigo

Los índices comienzan en 1. Ahora PO contiene el recorrido en postorden.

```
Función ReconstruirPostorden(EO, PO, i, j, p, padre)
    si i > j entonces
        devolver NONE

    m ← j - i + 1
    valorRaiz ← PO[p + m - 1]

    q ← i
    mientras EO[q]   valorRaiz hacer
        q ← q + 1

    tamañoIzquierdo ← q - i

    v ← CrearNodo(valorRaiz)
     (v) ← padre

    left(v) ← ReconstruirPostorden(
        EO, PO, i, q - 1, p, v
    )

    right(v) ← ReconstruirPostorden(
```

```
        EO, PO, q + 1, j, p + tamañoIzquierdo, v
    )

    devolver v
```

La llamada inicial es:

```
r ← ReconstruirPostorden(EO, PO, 1, n, 1, NONE)
```

3.10.2 Comprobación con un ejemplo

Consideremos los recorridos:

$$EO = [D, B, E, A, F, C, G],$$
$$PO = [D, E, B, F, G, C, A].$$

En la llamada inicial, $m = 7$ y $p = 1$. Por tanto, la raíz es $PO[7] = A$. Su posición en EO es $q = 4$, de modo que el subárbol izquierdo tiene tres vértices.

Los segmentos quedan separados así:

Subárbol	En orden	Postorden
Izquierdo de A	D, B, E	D, E, B
Derecho de A	F, C, G	F, G, C

La llamada izquierda utiliza $i = 1$, $j = 3$ y $p = 1$. Identifica como raíz a $PO[3] = B$, con D como hijo izquierdo y E como hijo derecho.

La llamada derecha utiliza $i = 5$, $j = 7$ y $p = 4$. Identifica como raíz a $PO[6] = C$, con F como hijo izquierdo y G como hijo derecho.

Así se reconstruye el árbol cuya raíz es A, con hijos B y C, y hojas D, E, F, G.

3.10.3 Justificación y consistencia del análisis

La prueba de terminación se conserva: la búsqueda encuentra la raíz dentro del segmento y cada llamada recursiva recibe un segmento estrictamente menor.

Para la corrección, se utiliza la misma inducción sobre el número de vértices, cambiando la identificación de la raíz y los segmentos de postorden. La raíz es el último elemento del segmento; sus primeros a elementos corresponden al subárbol izquierdo y los siguientes $m - 1 - a$ al derecho. Por tanto, las llamadas reciben exactamente los recorridos de los subárboles que deben reconstruir. La hipótesis inductiva garantiza su reconstrucción correcta y el algoritmo los enlaza con la raíz.

La búsqueda en EO sigue comenzando por la izquierda, por lo que el trabajo local continúa siendo $\Theta(a + 1)$. En consecuencia, se mantienen el tiempo $O(n^2)$ en el peor caso, el espacio auxiliar $O(h + 1)$ y el espacio $\Theta(n)$ del árbol producido.

4 Ejercicio 4. Consultas de rutas a gran escala

4.1 Enunciado

Ve el video indicado en la tarea y explica con tus propias palabras, en **no más de una página**, por qué el algoritmo de Dijkstra no es lo suficientemente rápido en sistemas del tamaño de empresas como Google y cómo puede modificarse para responder con rapidez a pesar de los grandes volúmenes de peticiones que reciben.

4.2 Explicación